

Project 1: Industrial PPE and Work-at-Height Safety Monitoring

Primary focus: AI/ML and edge computer vision

Project title: EdgeVision PPE Compliance and Work-at-Height Safety Platform

Business problem

Industrial organisations need to determine whether workers entering operational or hazardous areas are following required safety practices.

The system should identify:

PPE or safety item	Detection requirement
Safety helmet	Present, absent or incorrectly worn
Reflective safety vest	Present or absent
Safety boots	Present or absent
Safety harness/belt	Present or absent
Lanyard	Connected or disconnected
Safety hook	Present and connected appropriately
Worker	Person identification and tracking
Restricted zone	Unauthorised entry
Work at height	Worker operating above the configured height or platform zone

The main technical challenge is not merely detecting PPE objects. The system must determine whether the detected PPE belongs to the correct person.

For example:

Helmet detected in image
↓
Is the helmet positioned on a detected worker?
↓
Is that worker inside a restricted zone?
↓
Does the zone require a helmet, vest, boots or harness?
↓
Generate compliance or violation event

AI/ML scope

The candidate should develop a multi-stage computer-vision pipeline.

Stage 1: Person detection and tracking

Detect each worker and assign a temporary tracking ID.

Example:

Worker-101
Worker-102
Worker-103

Tracking should prevent the same worker from generating a new alert in every video frame.

Stage 2: PPE detection

Detect:

person
helmet
vest
boots
safety_belt
lanyard
hook
anchor_point

Helmet and vest detection will normally be easier than boots, safety belts, lanyards and hooks. Boots and hooks are often small, partially hidden or affected by camera angle.

Stage 3: Person-to-PPE association

The model must associate each detected item with the appropriate person.

Possible techniques include:

- Bounding-box containment
- Body-region mapping
- Human pose estimation
- Nearest-person association
- Tracking-based temporal association

For example:

Worker-101:
Helmet: Yes
Vest: Yes
Boots: Yes
Harness: No
Hook connected: No

Stage 4: Rule engine

Safety requirements should be configurable by zone.

Zone	Required PPE
General plant area	Helmet and vest
Construction area	Helmet, vest and boots
Work-at-height area	Helmet, vest, boots, harness and connected hook

Zone	Required PPE
Restricted machinery area	Helmet, vest and authorisation

Stage 5: Temporal validation

An alert should not be generated from a single uncertain frame.

Example rule:

Generate alert only when:
 Violation appears in 8 of the last 10 frames
 AND
 Confidence is above configured threshold
 AND
 The worker remains in the zone for more than 2 seconds

This will reduce false alerts caused by blur, partial visibility and temporary occlusion.

Improving model accuracy

The candidate should focus on the following areas:

Small-object detection

Boots, hooks and lanyards may occupy very few pixels. The candidate should evaluate:

- Higher inference resolution
- Person cropping before PPE detection
- Image tiling
- Small-object augmentation
- Camera-position-specific models
- Separate secondary detectors for difficult PPE classes

Difficult environmental conditions

The dataset should include:

- Low light
- Harsh sunlight
- Shadows
- Dust
- Motion blur
- Partial worker visibility
- Crowded scenes
- Different helmet and vest colours
- Workers facing away from the camera
- Workers bending or sitting
- PPE held in the hand instead of being worn

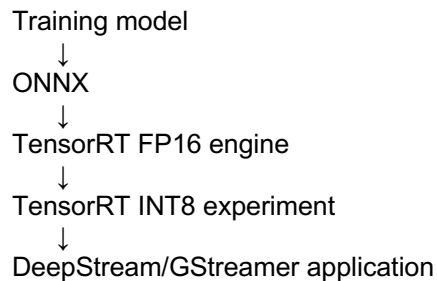
Hard-negative examples

The dataset must include situations such as:

- Yellow machinery incorrectly detected as a helmet
- Reflective material incorrectly detected as a vest
- Shoes incorrectly detected as safety boots
- Loose ropes incorrectly detected as lanyards
- Hooks lying nearby but not connected to the worker

Jetson deployment

The trained model should be exported through:



TensorRT supports optimisation of models imported from frameworks and ONNX, including reduced-precision inference. NVIDIA recommends creating a device-specific TensorRT engine for DeepStream deployment.

DeepStream supports custom detection and classification models, camera and RTSP inputs, and multi-stream GStreamer pipelines on Jetson. As of July 2026, DeepStream 9.1 includes Jetson Orin support; the project must nevertheless use the DeepStream version compatible with the JetPack/L4T installation on the actual device.

The TensorRT engine should be generated on the target Jetson or an identically configured device because engine compatibility depends on the deployment software and hardware environment.

Web application

Dashboard pages

Page	Function
Live Monitoring	Display live camera feed and detections
Active Violations	Show current safety violations
Event History	Search and review earlier incidents
Worker Compliance	Show compliance by worker or tracking ID
Zone Configuration	Configure required PPE by zone
Camera Management	Add and configure cameras
Reports	Daily, weekly and monthly safety reports

Page	Function
Model Monitoring	Display model version, FPS and accuracy statistics

Alert record

Each event should store:

Event ID
 Camera ID
 Zone ID
 Worker tracking ID
 Violation type
 Detected PPE
 Missing PPE
 Confidence
 Timestamp
 Image evidence
 Video clip
 Acknowledgement status
 Model version

Suggested database tables

cameras
 zones
 zone_ppe_rules
 ppe_types
 detection_events
 detected_objects
 worker_tracks
 violation_events
 alert_deliveries
 event_images
 event_videos
 model_versions
 inference_metrics
 users
 roles
 audit_logs

PostgreSQL is recommended for structured data, while images and video clips can be stored in file storage or object storage with their locations recorded in the database.

Model evaluation

The candidate must report:

Metric	Purpose
Precision per PPE class	Measures false detections

Metric	Purpose
Recall per PPE class	Measures missed PPE
mAP50 and mAP50–95	Overall object-detection accuracy
Violation precision	Whether generated violations are correct
False alerts per hour	Operational usefulness
FPS	Real-time performance
P95 inference latency	Worst-case response performance
GPU, CPU and memory use	Jetson resource utilisation
Temperature and power mode	Edge-device stability

Proposed initial deployment target

Single 1080p camera stream
 Minimum acceptable throughput: 12 FPS
 Preferred throughput: 20 FPS or higher
 FP16 TensorRT deployment
 Continuous operation for at least 8 hours
 No repeated alert for the same continuing violation

Targets may be revised after the first benchmark because hook and lanyard detection complexity will depend heavily on camera distance and dataset quality.

Final deliverable

A live camera should detect workers, identify required PPE, recognise a violation, record evidence in the database, display the event on the web application and generate an alert.

Mandatory Final Submission - candidate must submit:

1. Complete source code
2. Database creation and migration scripts
3. API documentation
4. Model-training code
5. Dataset structure and labelling guide
6. ONNX model
7. TensorRT engine-generation instructions
8. Jetson installation and startup service
9. Web application
10. Test cases
11. Accuracy report
12. FPS, latency, memory and temperature benchmark
13. User guide
14. Architecture diagram
15. Final recorded and live demonstration

The .engine file should not be treated as the only deployable asset. The ONNX model, calibration data, engine-generation command and exact Jetson software versions must also be maintained so the engine can be recreated.